

function named `draw()` which draws a triangle. Line 6 starts drawing the triangle. Line 7 sets the colour of the first vertex to red. In OpenGL, a vertex is a point in 3D space that has position denoted by (x, y, z) and other attributes such as colour, texture coordinates, and normal vectors. Line 8 sets the position of this vertex. Lines 9 to 12 do the same thing for the other two vertices of the triangle, with different colours and positions. After defining the vertices, the method `glEnd()` is called to stop drawing triangles (Line 13), and the method `glFlush()` is called to ensure that all commands are executed immediately (Line 14). Line 15 initialises the GLUT (OpenGL Utility Toolkit) library. Line 16 sets the size of the window to 250x250 pixels. Line 17 creates a window with the title 'Generative Art'. Line 18 sets the function `draw()`, defined earlier, to be called for rendering the scene. Finally, Line 19 starts the main loop that handles all the events and rendering. The triangle drawn by this Python script is shown in Figure 3.

Unfortunately, the fascinating field of audio processing did not receive much coverage in this series. Popular Python libraries such as Pydub, PyAudio, Librosa, and Kaldi are used for this purpose. Pydub is a library for audio processing that simplifies the processing of audio files in various formats. It offers features such as slicing, concatenating, filtering, and effects like fading and normalising. Pydub is widely used for tasks such as audio file conversion, editing, and creating snippets for music applications. To install Pydub, execute the command `pip3 install pydub` in the terminal. Now, let us consider the Python script shown below (line numbers are included for ease of explanation) which utilises Pydub to read an Ogg file (the audio format used by Wikipedia).

```
1. import pydub
2. import numpy as np
```

```
3. import matplotlib.pyplot as plt
4. audio_file = pydub.AudioSegment.
  from_file('file.ogg', format='ogg')
5. audio_data = np.array(audio_file.
  get_array_of_samples())
6. plt.plot(audio_data)
7. plt.show()
```

Let us go through the code line by line. Lines 1 to 3 import the necessary libraries. Line 4 uses the method `from_file()` of the class `AudioSegment` to read the audio data from the Ogg audio file named `file.ogg`. The parameter 'format' is used to specify the format of

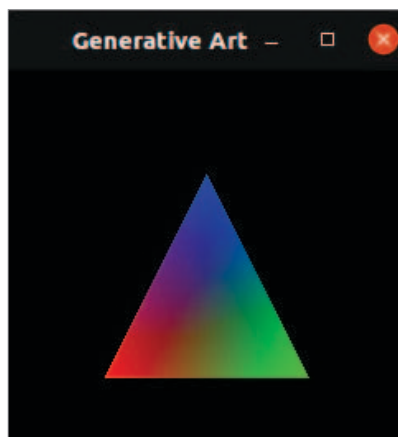


Figure 3: A triangle drawn with PyOpenGL

the audio file as Ogg. Ogg is a free and open format for audio and video which is unrestricted by software patents. Line 5 uses the method `get_array_of_samples()` of the class `AudioSegment` to extract the audio data as an array of samples and convert it to a NumPy array using the function `array()`. Each sample represents the amplitude of the audio signal at a particular point in time. Lines 6 and 7 create a line plot of this audio data and plot it. Figure 4 shows the audio signal generated from the audio clip. I have used the audio clip of the first moon landing in 1969 (Courtesy: Wikimedia Commons). The audio also contains the now-famous quote, 'that's one small step for man, one giant leap for mankind', by Neil Armstrong. There is a lot of static noise in the original audio clip which results in the specific audio signal we obtained. After downloading another audio file in Ogg format without any static noise, rerun the code to see the change in the corresponding audio signal.

The fourth article in this series discussed parallelising Python code using Numba. However, what about concurrent programming in Python? To understand the difference between

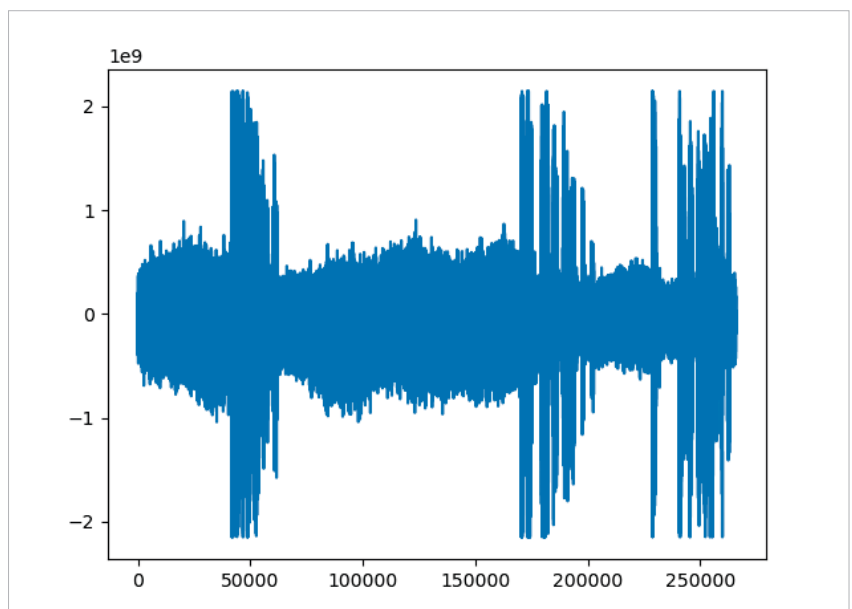


Figure 4: Output of the Python script using Pydub